

Proyecto Integrador

Biblioteca Horizonte

Equipo Biblioteca Horizonte	Joaquin Pereyra Santiago Torretta Santiago Bruna Geronimo Dominguez

Institución	Tecnicatura Superior en Desarrollo de Software · Sede Río Tercero
Materia	Ingeniería de Software — Proyecto Integrador
Versión / fecha	1.0 · 2 de octubre de 2026

Qué es este documento. Reúne en un solo lugar la ficha del proyecto, los requisitos, el backlog, el diseño, la organización del equipo, las pruebas, la calidad y la seguridad de Biblioteca Horizonte. Describe el sistema **tal como está hoy**: lo que funciona, lo que está parcial y lo que queda como mejora futura. El proyecto es un prototipo académico de demostración.

1. Ficha del proyecto

Producto	Biblioteca Horizonte — Sistema de reservas de equipos
Problema que resuelve	La Biblioteca Escolar Horizonte presta 2 proyectores y 4 notebooks a docentes. Los pedidos llegan por mensajes y se anotan en un cuaderno, lo que genera confusiones y, a veces, dos personas creen tener reservado el mismo equipo.
Objetivo del MVP	Que un docente identificado solicite un equipo para una fecha y un módulo horario, y que Lucía (bibliotecaria) pueda confirmar o rechazar cada solicitud, sin que existan dos reservas CONFIRMADAS para el mismo equipo, fecha y módulo.
Regla central	Solo puede haber 1 reserva CONFIRMADA por equipo + fecha + módulo. Una solicitud PENDIENTE no garantiza disponibilidad y no bloquea el equipo.

Alcance inicial (incluye)

- Inicio de sesión con correo y contraseña para docentes y bibliotecaria.
- Listado de equipos activos (2 proyectores y 4 notebooks).
- Solicitud de un equipo para una fecha y un módulo horario completo; la solicitud nace en estado PENDIENTE.
- Panel de Lucía para ver, confirmar y rechazar solicitudes (rechazo con motivo).
- Validación en el servicio que impide una segunda confirmación para el mismo equipo, fecha y módulo.
- Vista del docente con el estado real de sus solicitudes (pendiente, confirmada, rechazada).

Fuera de alcance

- Pagos, multas, inventario de reparaciones y reservas recurrentes (excluidos por el caso original).

Recordatorios por correo electrónico: pedido de Lucía registrado como cambio y postergado al backlog (ver sección 12).

- Reservas por franjas parciales (solo módulos completos).
- Autenticación con JWT y contraseñas cifradas, pruebas end-to-end e integración continua: mejoras futuras (ver secciones 4 y 14).

Supuestos

- Hay una sola bibliotecaria (Lucía) que gestiona todas las solicitudes.
- Los módulos horarios son 4 fijos por día (Módulo 1 a Módulo 4) y son iguales para todos los equipos.
- Los docentes ya tienen una cuenta cargada; no hay registro autogestionado.
- La demostración se hace en una computadora local; los datos de prueba se cargan al iniciar el sistema.

Riesgos iniciales

Pantalla y servicio avanzan sin acordar datos ni mensajes	Interfaz que dice “reservado” cuando el servicio tiene PENDIENTE (como en el caso de clase)	Contrato de integración escrito (sección 7.4) y estados con nombres idénticos en ambas capas
Confirmaciones duplicadas	Dos docentes creen tener el mismo equipo	Regla validada en el servicio; restricción en base de datos como mejora
Trabajo aislado por integrante	Partes que no integran	Revisión cruzada obligatoria y versión integrada ejecutable
Alcance que crece (recordatorios)	MVP sin terminar	Gestión de cambios: evaluar y postergar

2. Stakeholders

Lucía — bibliotecaria	Gestionar solicitudes y evitar duplicaciones.	Confirmar o rechazar solicitudes; pedir cambios (recordatorios).	Reglas de reserva y el flujo real de trabajo.
Docente solicitante	Pedir un equipo y saber en qué estado está su pedido.	Crear solicitudes y consultar las propias.	Claridad del proceso y de los mensajes (pendiente ≠ confirmada).
Institución (escuela)	Uso ordenado de recursos y trazabilidad.	Define políticas generales de préstamo.	Que el sistema respete las reglas institucionales.
Equipo de desarrollo	Un producto demostrable y entregable.	Decide tecnología, diseño y prioridades técnicas.	Que lo entregado cumpla la consigna y la DoD.
Cátedra (docentes de la materia)	Evidencias de proceso, calidad y trabajo colaborativo.	Evaluar la entrega según la rúbrica.	Que el proyecto cumpla las condiciones mínimas.

3. Equipo, roles y acuerdo de trabajo

3.1 Integrantes y roles

Joaquin Pereyra	Frontend + UX	Manual de usuario	Pantallas, estados visuales, mensajes al docente y a Lucía.
Santiago Torretta	Backend + base de datos	Integración	API, reglas de reserva, persistencia y datos iniciales.
Santiago Bruna	Pruebas y calidad	Seguridad	Casos de prueba, registro de defectos, análisis de riesgos.
Geronimo Dominguez	Coordinación y documentación	Control de versiones	Backlog, bitácora de decisiones, repositorio Git y documento final.
Los cuatro	Revisión cruzada	Definition of Done	Ninguna tarea se cierra sin que otro integrante la revise.

3.2 Acuerdo de trabajo

Canal principal	Grupo de chat del equipo; las decisiones importantes se copian a la bitácora del repositorio.
Bloqueos	Se informan con el formato HECHO + EFECTO + AYUDA NECESARIA + PRÓXIMO PASO, el mismo día.
Coordinación	Reunión breve al comenzar cada jornada de trabajo: qué se hizo, qué sigue, qué bloquea.
Decisiones	Se registran con fecha, quién decidió y motivo (cambios de reglas, de contrato o de alcance).
Revisión de cambios	Cambios en ramas propias; se integran a la rama principal después de la revisión de otro integrante.
Desacuerdos técnicos	Se discuten con evidencia (prueba, ejemplo o documentación); si no hay acuerdo, se vota y se registra la decisión.
Convención de commits	Prefijo + mensaje en imperativo: feat:, fix:, docs:, test:, refactor: (ej. "fix: validar estado antes de confirmar").
Convención de ramas	main estable; ramas feature/<tema> y fix/<tema> por tarea.

3.3 Definition of Done (DoD)

- Responde a la historia seleccionada y cumple sus criterios de aceptación.
- Está integrada en la rama compartida y la revisó otro integrante.
- Se ejecutaron las pruebas pertinentes y no hay defectos críticos conocidos.
- La documentación necesaria está actualizada.
- Puede demostrarse en una versión que otro integrante pueda ejecutar.

4. Arquitectura y tecnologías

Arquitectura cliente-servidor simple: una aplicación web (frontend) que consume una API REST (backend) con persistencia en base de datos. Es suficiente para el alcance acotado y la demostración.

Frontend	React + TypeScript + Vite	Rápido de levantar, muy documentado y con tipado que ayuda a respetar el contrato de la API.

Estilos	CSS propio (sin Tailwind)	Control total del diseño (ficha de préstamo, sellos de estado) sin depender de un compilador de estilos.
Backend	Java 21 + Spring Boot	Framework conocido en la cursada; API REST con poca configuración.
Persistencia	Spring Data JPA	Mapea las entidades a tablas y permite consultas simples por nombre de método.
Base de datos	H2 en memoria	No requiere instalar un servidor; los datos iniciales se cargan al arrancar. Los datos se pierden al reiniciar.
Autenticación	Login simple por correo y contraseña (/api/auth/login)	Suficiente para identificar al docente en el prototipo. Sus límites están en la sección 11.
Pruebas	JUnit 5 + MockMvc (opcional) y casos manuales documentados	JUnit permite repetir los casos como regresión; los manuales alcanzan para la demostración.
Control de versiones	Git	Historial, ramas y revisión entre integrantes.
Documentación	Este documento y README de cada proyecto	Simple y versionable en el repositorio.

Mejoras futuras (no incluidas en esta versión)

- Autenticación con JWT y contraseñas cifradas con bcrypt.
- Pruebas end-to-end con Playwright e integración continua con GitHub Actions.
- Base de datos persistente (PostgreSQL) y documentación de la API con OpenAPI.

Estructura de la solución

Backend	biblioteca-horizonte/	model (entidades y enums), repository (JPA), service (reglas de reserva), controller (API REST), config (datos iniciales).
Frontend	biblioteca-frontend-v2/	pages (Docente, Bibliotecaria), components, services/api.ts (cliente HTTP), types.ts (tipos compartidos), context (avisos).

5. Requisitos

5.1 Requisitos funcionales

RF01	Identificar al docente que solicita un recurso.	HU-01	Implementado (login simple)
RF02	Consultar los recursos que pueden reservarse.	HU-02	Implementado
RF03	Crear una solicitud indicando recurso, fecha y módulo.	HU-03	Implementado; validaciones parciales
RF04	Registrar la solicitud inicialmente como pendiente.	HU-03, HU-10	Implementado
RF05	Permitir a Lucía consultar solicitudes pendientes.	HU-05	Implementado
RF06	Permitir a Lucía confirmar una solicitud.	HU-06	Implementado
RF07	Impedir 2 confirmaciones para el mismo recurso, fecha y módulo.	HU-08	Implementado en el servicio; falta restricción en la base
RF08	Permitir a Lucía rechazar una solicitud.	HU-07	Implementado (motivo opcional)

RF09	Permitir al docente conocer el estado de su solicitud.	HU-04	Implementado
RF10	Consultar reservas confirmadas por recurso y fecha.	HU-11	Pendiente (backlog)

5.2 Requisitos no funcionales (verificables)

RNF01	Usabilidad	El estado de una solicitud se muestra con una etiqueta y un texto inequívoco; una solicitud pendiente nunca se presenta como confirmada.	Revisión visual de las 3 etiquetas y mensaje al crear la solicitud.
RNF02	Confiabilidad	Ante una segunda confirmación para el mismo equipo, fecha y módulo, el sistema la rechaza y mantiene la primera.	Caso de prueba CP-08.
RNF03	Seguridad	Las operaciones de confirmar y rechazar solo las puede realizar el rol BIBLIOTECARIA, validado en el servidor.	Pruebas de autorización CP-12 (hoy es una limitación conocida).
RNF04	Seguridad	La API no devuelve contraseñas ni información interna en sus respuestas.	Inspección de respuestas de login y listados (CP-13).
RNF05	Mantenibilidad	Las reglas de reserva están en la capa de servicio, separadas del controlador y de la interfaz.	Revisión de código: la regla vive en ReservaService.
RNF06	Portabilidad	El sistema se ejecuta en otra computadora con Java 21 y Node 18+ siguiendo el README, sin instalar un servidor de base de datos.	Ejecución de la sección 13 en otro equipo.

6. Historias de usuario y Product Backlog

6.1 Product Backlog

HU-01	Como docente, quiero iniciar sesión para que mi solicitud quede identificada.	Alta	3	—	Hecho (básico)
HU-02	Como docente, quiero ver los equipos disponibles para elegir uno.	Alta	2	HU-01	Hecho
HU-03	Como docente, quiero solicitar una reserva para una fecha y módulo.	Alta	3	HU-01, HU-02	Hecho
HU-04	Como docente, quiero ver el estado de mis solicitudes.	Alta	3	HU-03	Hecho
HU-05	Como Lucía, quiero ver las solicitudes pendientes.	Alta	2	HU-03	Hecho
HU-06	Como Lucía, quiero confirmar una solicitud.	Alta	5	HU-05	Hecho
HU-07	Como Lucía, quiero rechazar una solicitud indicando un motivo.	Media	2	HU-05	Hecho
HU-08	Como Lucía, quiero que el sistema no me deje confirmar dos reservas para el mismo equipo, fecha y módulo, para que dos docentes no crean tener el mismo equipo.	Alta	5	HU-06	Parcial (servicio sí, base no)
HU-09	Como Lucía, quiero recibir recordatorios por correo.	Baja	8	HU-06, servidor de correo	Postergada

HU-10	Como docente, quiero poder solicitar un equipo aunque haya solicitudes pendientes, porque pendiente no bloquea.	Alta	1	HU-03	Hecho
HU-11	Como Lucía, quiero consultar las reservas confirmadas por equipo y fecha.	Media	3	HU-06	Pendiente
HU-12	Como Lucía, quiero que solo yo pueda confirmar o rechazar, para que un docente no altere reservas.	Alta	5	HU-01	Pendiente
HU-13	Como docente, quiero mensajes claros cuando los datos son inválidos (fecha pasada, equipo inexistente).	Alta	3	HU-03	Parcial
HU-14	Como institución, quiero saber quién confirmó o rechazó cada reserva y cuándo.	Media	3	HU-06, HU-07	Pendiente
HU-15	Como institución, quiero que las contraseñas se guarden cifradas y la sesión expire.	Media	8	HU-01	Mejora futura

6.2 Historias completas con criterios de aceptación

HU-01 — Iniciar sesión

Como docente, quiero iniciar sesión para que mi solicitud quede identificada.

- Con correo y contraseña válidos, el sistema me lleva al panel que corresponde a mi rol.
- Con datos incorrectos muestra un error genérico (“Correo o contraseña incorrectos”) sin decir cuál falló.
- Al cerrar sesión vuelvo a la pantalla de ingreso.

HU-03 — Solicitar una reserva

Como docente, quiero solicitar un equipo para una fecha y un módulo para saber si podré usarlo en mi clase.

- Con equipo, fecha y módulo válidos, la solicitud se crea en estado PENDIENTE.

- La pantalla informa que quedó pendiente (no “reservado” ni “confirmado”).
- Puedo solicitar aunque ya existan solicitudes pendientes para ese equipo, fecha y módulo.
- Si falta algún dato, no se envía la solicitud.

HU-04 — Ver el estado de mis solicitudes

Como docente, quiero ver el estado de mis solicitudes para saber cuáles están confirmadas.

- Veo solo mis solicitudes, ordenadas por fecha.

- Cada una muestra equipo, fecha, módulo y una etiqueta de estado: pendiente, confirmada o rechazada.
- Si fue rechazada, veo el motivo cuando Lucía lo escribió.
- Si no tengo solicitudes, se muestra un mensaje que lo explica.

HU-06 — Confirmar una solicitud

Como Lucía, quiero confirmar una solicitud para asegurar el equipo a un docente.

- Al confirmar una solicitud pendiente, su estado pasa a CONFIRMADA y el docente lo ve.
- Si ya existe otra CONFIRMADA para el mismo equipo, fecha y módulo, el sistema rechaza la confirmación y muestra el motivo.
- La primera reserva confirmada no se modifica.

HU-07 — Rechazar una solicitud

Como Lucía, quiero rechazar una solicitud indicando un motivo para que el docente sepa por qué.

- Al rechazar, el estado pasa a RECHAZADA.

- El motivo es opcional y, si se escribe, se guarda y se muestra al docente.
- Una solicitud rechazada deja de aparecer entre las pendientes.

HU-08 — No permitir confirmaciones duplicadas

Como Lucía, quiero que el sistema impida dos confirmaciones para el mismo equipo, fecha y módulo.

- El servicio valida la existencia de una CONFIRMADA antes de confirmar.
- La prueba que intenta duplicar debe fallar con un mensaje claro (CP-08).
- Mejora planificada: reforzar con una restricción en la base de datos y ejecución transaccional.

7. Diseño

7.1 Flujo del proceso

Docente inicia sesión -> elige equipo -> elige fecha y módulo -> se crea la solicitud en estado **PENDIENTE** -> Lucía revisa las pendientes -> **confirma** (si no hay otra confirmada para ese equipo, fecha y módulo) o **rechaza** con motivo -> el docente consulta el estado final.

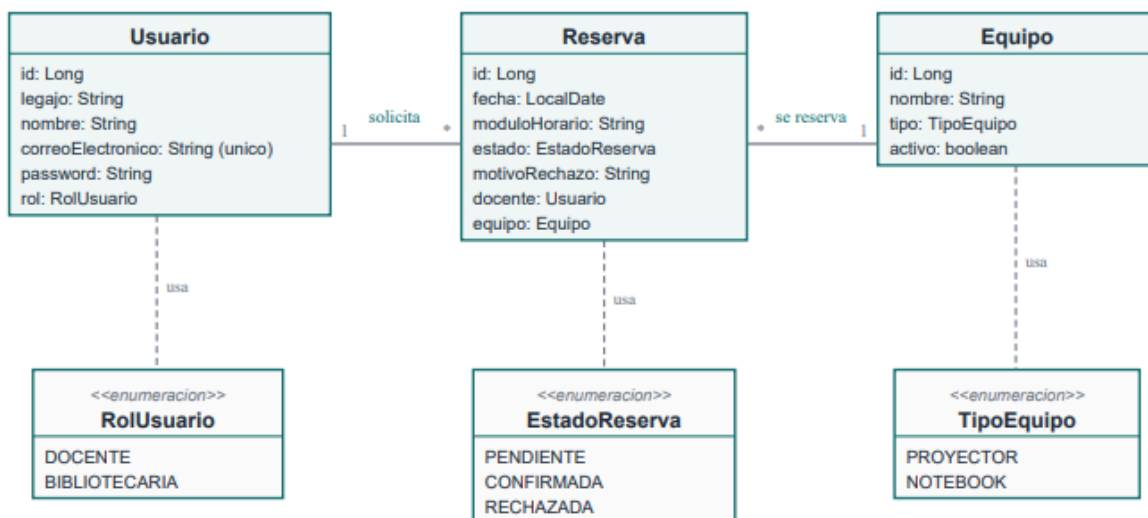
7.2 Estados, transiciones y mensajes

PENDIENTE	Docente al solicitar	A CONFIRMADA o RECHAZADA, solo por Lucía	"Solicitud registrada como pendiente". Pendiente no significa confirmada.
CONFIRMADA	Lucía	Final (sin cambios posteriores)	Etiqueta CONFIRMADA: el equipo está asegurado.
RECHAZADA	Lucía	Final (sin cambios posteriores)	Etiqueta RECHAZADA y el motivo, si existe.

Regla de no duplicación: antes de confirmar, el servicio consulta si existe otra reserva CONFIRMADA con el mismo equipo, fecha y módulo; si existe, no confirma y devuelve un error con el motivo. **Datos inválidos u operación no permitida:** el servicio responde con un mensaje de error que la interfaz muestra como aviso.

Limitación conocida: las transiciones "finales" de la tabla son la regla de diseño, pero hoy el servicio no impide, por ejemplo, confirmar una solicitud ya rechazada (defecto D-01, sección 9.2).

7.3 Modelo de datos



7.4 Contrato de integración (frontend ↔ backend)

Base: <http://localhost:8080/api> (configurable en el frontend con VITE_API_URL). Los estados viajan como

texto exacto: PENDIENTE, CONFIRMADA, RECHAZADA.

Método y ruta	Quién la usa	Envía	Devuelve
POST /auth/login	Todos	{ email, password }	Usuario (200) o texto de error (401)
GET /equipos	Docente	—	Lista de equipos activos
GET /reservas?docenteld=	Docente / Lucía	docenteld opcional	Lista de reservas (solo las del docente si se indica)
POST /reservas	Docente	{ equipo:{id}, docente:{id}, fecha, moduloHorario }	Reserva creada con estado PENDIENTE
PATCH /reservas/{id}/confirmar	Lucía	—	Reserva CONFIRMADA (200) o mensaje de error (400)
PATCH /reservas/{id}/rechazar	Lucía	{ motivo }	Reserva RECHAZADA

Ejemplo de solicitud y respuesta:

```
POST /api/reservas
{ "equipo": { "id": 1 }, "docente": { "id": 2 }, "fecha": "2026-10-15", "moduloHorario": "Módulo 2" }

200 OK
{ "id": 7, "fecha": "2026-10-15", "moduloHorario": "Módulo 2", "estado":
"PENDIENTE", "motivoRechazo": null, ... }
```

Rechazo por duplicado:

```
PATCH /api/reservas/8/confirmar
400 Bad Request
"Error: Ya existe una reserva CONFIRMADA para este equipo, fecha y módulo."
```

7.5 Decisiones técnicas

Dónde se aplica la regla de no duplicación	En ReservaService.confirmarReserva, con la consulta existsByEquipoldAndFechaAndModuloHorarioAndEstado. Mejora: restricción en la base y transacción.
Persistencia	JPA sobre H2 en memoria, con tablas creadas al iniciar (create-drop) y datos de prueba cargados por DataInitializer.
Separación de responsabilidades	Controlador (HTTP) > servicio (reglas) > repositorio (datos). En el frontend: páginas, componentes y un único cliente HTTP (services/api.ts).
Ejecución en otra computadora	Ver sección 13 (Java 21, Node 18+, sin servidor de base de datos).

8. Proceso de desarrollo

El trabajo se organizó en cuatro iteraciones (sprints) con el objetivo de ir sumando valor sobre una versión integrada. Por tratarse de un prototipo académico, las iteraciones fueron cortas y el seguimiento fue liviano; este cuadro resume qué se buscó y en qué estado quedó cada una.

1	Que un docente pueda solicitar y ver que queda pendiente.	Login, listado de equipos, creación de solicitud y panel del docente.	Completado

2	Que Lucía pueda confirmar y rechazar.	Panel de Lucía, confirmar, rechazar con motivo y regla de no duplicación en el servicio.	Completado
3	Completar el recorrido de punta a punta.	Estados visibles, contadores e historial en el panel de Lucía, avisos y estados vacíos.	Completado
4	Calidad, seguridad y documentación.	Este documento, análisis de riesgos y defectos conocidos. Endurecimiento del código queda en el backlog.	Parcial

Control de versiones

- El código se mantiene en un repositorio Git compartido, con la convención de commits y ramas de la sección 3.2.
- Los zips de entrega no deben incluir target/, node_modules/ ni .idea/; se excluyen con .gitignore.
- No se suben contraseñas, tokens ni claves. Las credenciales de la demostración son datos de prueba.

9. Pruebas

9.1 Casos de prueba

CP-01	HU-01	Login con docente@horizonte.com y contraseña correcta	Ingresa al panel docente	Completar al ejecutar	A ejecutar
CP-02	HU-01	Login con contraseña incorrecta (negativo)	Error genérico, no ingresa	Completar al ejecutar	A ejecutar
CP-03	HU-02	Docente abre su panel	Lista 2 proyectores y 4 notebooks	Completar al ejecutar	A ejecutar
CP-04	HU-03	Docente + Proyector 1 + fecha futura + Módulo 1	Se crea solicitud PENDIENTE	Completar al ejecutar	A ejecutar
CP-05	HU-03	Enviar sin fecha (negativo)	No se envía; la interfaz pide la fecha	Completar al ejecutar	A ejecutar

CP-06	HU-03	Solicitud con equipo inexistente, vía API (negativo)	Rechazo con error claro	Completar al ejecutar	A ejecutar
CP-07	HU-10	Dos docentes piden el mismo equipo, fecha y módulo	Ambas quedan PENDIENTES	Completar al ejecutar	A ejecutar
CP-08	HU-06 / HU-08	Confirmar una; luego intentar confirmar la otra para el mismo equipo, fecha y módulo	La segunda se rechaza con mensaje; la primera sigue CONFIRMADA	Completar al ejecutar	A ejecutar
CP-09	HU-06	Confirmar una solicitud pendiente	Pasa a CONFIRMADA y el docente lo ve	Completar al ejecutar	A ejecutar
CP-10	HU-07	Rechazar con motivo	Pasa a RECHAZADA y el docente ve el motivo	Completar al ejecutar	A ejecutar

CP-11	HU-04	Docente consulta sus solicitudes	Ve solo las suyas, con etiqueta correcta	Completar al ejecutar	A ejecutar
CP-1 2	HU-12	Un docente llama a /confirmar por API (autorización)	Debe responder 403 (hoy: lo permite, ver D-04)	Completar al ejecutar	A ejecutar
CP-1 3	RNF04	Inspeccionar respuesta de login y de /reservas	No incluye contraseñas (hoy: sí las incluye, ver D-02)	Completar al ejecutar	A ejecutar
CP-1 4	HU-06	Confirmar una solicitud ya RECHAZADA (negativo)	Debe rechazarse la operación (hoy: la permite, ver D-01)	Completar al ejecutar	A ejecutar
CP-1 5	D-01	Regresión: repetir CP-08 después de corregir D-01	La regla de no duplicación sigue vigente	Completar al ejecutar	A ejecutar
CP-1 6	D-02	Regresión: repetir CP-01 y CP-09 después de corregir D-02	Login y confirmación siguen funcionando sin devolver contraseñas	Completar al ejecutar	A ejecutar

9.2 Registro de defectos conocidos

D-01	Rechazar una solicitud y luego llamar a /confirmar sobre ella.	No debe confirmarse una solicitud que no está pendiente.	Pasa a CONFIRMADA.	Alta	Abierto
D-02	Hacer login o consultar /reservas.	No devolver la contraseña.	La respuesta incluye el campo password en texto plano.	Crítica	Abierto
D-03	Rechazar una reserva con un id inexistente.	Error 404 claro.	Error 500 sin mensaje útil.	Media	Abierto
D-04	Llamar a /confirmar o /rechazar sin ser bibliotecaria.	403 Prohibido.	La operación se ejecuta.	Alta	Abierto
D-05	Crear una solicitud con fecha pasada o equipo inactivo, por API.	Error de validación.	Se crea la solicitud.	Media	Abierto

10. Calidad

Adecuación funcional	El sistema debe cubrir el flujo y la regla central.	Requisitos funcionales implementados / total (RF01–RF10).	9 de 10 implementados; RF10 pendiente y RF07 sin restricción en la base.
Usabilidad	Pendiente y confirmada no deben confundirse.	Cantidad de estados con etiqueta y texto propios.	3 de 3 estados con etiqueta distinta; mensaje “pendiente” al solicitar.
Confiabilidad	No pueden existir reservas confirmadas contradictorias.	Casos negativos y de regresión que pasan.	Regla validada en el servicio; falta cerrar D-01 y la restricción en base.

Seguridad	Solo la bibliotecaria debe alterar reservas.	Operaciones sensibles protegidas por rol / total.	0 de 2 protegidas hoy (D-04); es la prioridad de mejora.
Mantenibilidad	Las reglas deben poder cambiar sin romper todo.	Reglas ubicadas en la capa de servicio.	La regla central está aislada en ReservaService.

Interpretación: el producto cumple el recorrido principal y la regla central en condiciones normales. Lo que aún no puede sostenerse con evidencia es la seguridad y la robustez ante usos indebidos, por eso el backlog futuro empieza por autorización, estados válidos y restricción en base de datos.

11. Seguridad

Se usan los conceptos de activo, amenaza, vulnerabilidad, impacto y control, con el OWASP Top 10:2025 como referencia donde corresponde.

Reservas	Docente intenta confirmar o rechazar sin permiso	Sin control de rol en el servidor (control de acceso)	Reservas alteradas o duplicadas	Validar rol BIBLIOTECARIA en el servidor	Pendiente (D-04)
Cuentas	Robo de contraseñas	Contraseñas en texto plano y devueltas en la respuesta	Suplantación de docentes y de Lucía	DTO sin contraseña; bcrypt	Pendiente (D-02)
Solicitudes	Pedir a nombre de otro docente	El docenteld lo envía el cliente	Solicitudes falsas	Tomar el docente de la sesión	Pendiente
Datos de entrada	Datos inválidos o inesperados	Validación mínima en el servicio	Reservas inconsistentes	Validar fecha, equipo y módulo	Parcial (D-05)
Sistema	Errores que revelan información interna	Excepciones sin tratar (500)	Fuga de detalles técnicos	Manejo de errores centralizado	Pendiente (D-03)
Configuración	Acceso desde cualquier origen	CORS abierto a "*" y consola H2 habilitada	Uso indebido de la API y la base	Restringir orígenes y desactivar consola fuera de la demo	Pendiente
Historial	No poder reconstruir quién hizo qué	No se registra quién confirmó ni cuándo	Sin trazabilidad	Registrar usuario y fecha de cada cambio (HU-14)	Pendiente

12. Gestión de cambios, comunicación y mejora

12.1 Registro de cambios: recordatorios por correo

Cambio solicitado	Lucía pidió recibir recordatorios por correo electrónico.
Problema o beneficio	Evitar olvidos de solicitudes pendientes y de fechas próximas.
Esfuerzo y dependencias	8 puntos. Requiere servidor de correo, plantillas y una tarea programada; el correo del usuario ya existe en el modelo.

Impacto en MVP	No aporta a la regla central y compite con tareas de mayor prioridad (seguridad y validaciones).
Decisión	Postergar al backlog como HU-09 (prioridad baja). Si se acepta a futuro, hay que actualizar historia, criterios, diseño y pruebas.

12.2 Comunicación de bloqueos

Formato acordado: **HECHO + EFECTO + AYUDA NECESARIA + PRÓXIMO PASO**. Ejemplo: “La pantalla envía la solicitud pero el servicio devuelve un estado con otro nombre. No puedo integrar el recorrido. Necesito revisar el contrato de respuesta con quien mantiene el servicio; propongo hacerlo antes de integrar la rama.”

12.3 Mejoras al proceso

Tareas marcadas como terminadas sin prueba asociada.	Exigir en la DoD un revisor y un caso de prueba ejecutado.	Porcentaje de tareas cerradas con revisor y prueba.
Reglas del servicio sin pruebas automáticas que detecten regresiones.	Agregar pruebas JUnit para confirmar, rechazar y duplicado.	Cantidad de casos de la sección 9 repetibles con un solo comando.
Zips de entrega con carpetas generadas (target, node_modules).	Configurar .gitignore y entregar desde el repositorio.	El zip pesa pocos MB y no contiene carpetas generadas.

13. Cómo ejecutar el proyecto y demostración

Requisitos: Java 21 y Node 18 o superior.

1) **Backend** (carpeta biblioteca-horizonte)
`./mvnw spring-boot:run`
Queda en `http://localhost:8080` (consola de la base en `/h2-console`).

2) **Frontend** (carpeta biblioteca-frontend-v2)
`npm install`
`npm run dev`
Queda en `http://localhost:5173`.

ROL	Correo	Contraseña
Docente	docente@horizonte.com	1234
Bibliotecaria (Lucía)	lucia@horizonte.com	1234

Guión de la demostración

- El docente inicia sesión y crea una solicitud válida; queda PENDIENTE.
- Lucía inicia sesión, ve la solicitud pendiente y la confirma.
- El docente consulta y ve el estado CONFIRMADA.
- Se crea otra solicitud para el mismo equipo, fecha y módulo; al intentar confirmarla, el sistema la rechaza (regla central).
- Lucía rechaza otra solicitud con un motivo y el docente lo ve.
- Se muestran un caso de prueba (CP-08) y una decisión de seguridad (sección 11), y el historial de Git.

14. Cierre: deuda técnica, backlog futuro y conclusiones

14.1 Deuda técnica y backlog futuro (en orden de prioridad)

- **Seguridad primero:** autorización por rol en el servidor (HU-12), no devolver contraseñas (D-02) y luego bcrypt + JWT con expiración (HU-15).
- **Integridad de la regla central:** validar que solo se confirmen solicitudes pendientes (D-01), ejecutar la

confirmación en una transacción y agregar restricción en la base de datos (HU-08).

- **Validaciones y errores:** fecha pasada, equipo inactivo o inexistente y respuestas 400/404/409 claras (D-03, D-05, HU-13).
- **Funcionalidad:** consulta de confirmadas por equipo y fecha (HU-11), auditoría de quién confirmó (HU-14) y entidad Módulo.
- **Infraestructura:** PostgreSQL, pruebas JUnit/Playwright y CI en GitHub Actions; recordatorios por correo (HU-09).

14.2 Estado frente a las condiciones mínimas de aprobación

El sistema ejecuta y puede demostrarse	Cumple (sección 13).
La regla de no duplicar está implementada y probada	Implementada en el servicio; prueba CP-08 a ejecutar y registrar.
Pendiente y confirmada no se presentan como equivalentes	Cumple (etiquetas y mensajes distintos).
El repositorio muestra trabajo colaborativo	Se respalda con el historial de Git del equipo (adjuntar captura).
Requisitos relacionados con historias y pruebas	Cumple (secciones 5, 6 y 9).
Evidencias de calidad y seguridad	Documentadas en las secciones 10 y 11; los controles pendientes están identificados.
Todos pueden explicar el producto y el proceso	Cada integrante conoce su rol y la revisión cruzada (sección 3).

14.3 Conclusiones y aprendizajes

El principal aprendizaje del caso fue que dividir las tareas no alcanza: la pantalla y el servicio tienen que acordar estados, datos y mensajes antes de integrarse, y una función no está terminada hasta que otra persona la revisó y se probó. También quedaron claras las limitaciones de un prototipo: lo que funciona en una demostración todavía necesita controles de seguridad y de integridad antes de usarse con usuarios reales. El backlog futuro deja ese camino ordenado y verificable.